

이름 : 학번:

솔루션 : SolTypeA_학번 (SolTypeA_123456)

프로젝트 1 :ProjNo1 (Console App)

프로젝트 2 :ProjNo2 (Console App)

제출은 반드시 저장한다. 저장 후 bin, obj 폴더를 삭제하고, 솔루션 폴더를 압축하여 제출한다. 해당 시험지는 반드시 제출해야 한다.

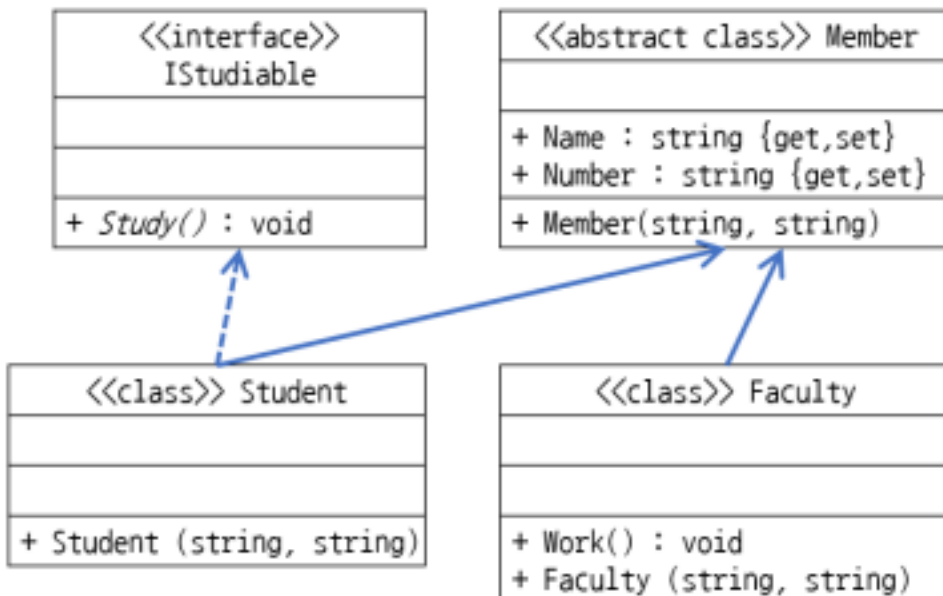
[UML 표기]

- : private | # : protected | + : public | underbar : static | italic : abstract

[프로젝트 1]

관계만 정확하게 구성하면 됨.

메소드 내용은 구현하지 않고, 빈 블록으로 처리할 것 반환타입이 있는 경우 잘 알려진 기본 값으로 반환할 것 매개변수는 주어진 표를 보고 유추해볼 것



[프로젝트 2]

배포한 실행 프로그램을 실행해보고

뒤의 UML과 Main()를 참고하여 프로그램을 작성한다.

프로그램을 실행하면 c:\finalA\students.dat 파일이 생성된다.

파일을 열어보면 학번|이름|학과|성적 0,...,성적 n 형태로 이루어진 내용이

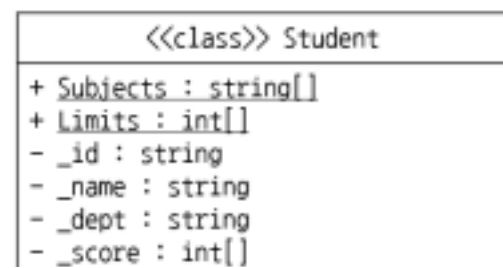
있다.

```
1|John|CS|26,14,12,7
2|Mary|EE|29,6,14,13
3|Tom|ME|5,14,10,10
```

[프로젝트 2]

Student : 학생의 성적을 관리하는 클래스

- Subjects : 입력할 성적 과목 목록 (Main() 참고)



- Limits : 입력할 성적 과목의 최고점 목록 (Main() 참고)
- _id / Id : 학번
- _name / Name : 이름
- _dept / Dept : 학과
- _score : 입력한 과목별 성적 목록
- Sum : 입력한 과목별 성적 목록의 점수 합산
합산할 성적이 없는 경우는 0
- Record : 파일에 저장할 학생 정보의 문자열 (생성 파일 참고)
- 생성자 : 학번, 이름, 학과를 받아서 설정하고,
입력해야 할 과목 목록을 참고하여 _score를 생성한다.
- SetScore : 과목 순번과 점수를 받아 _score에 설정한다.
단, 점수가 0점 미만이거나 Limits에 설정한 점수보다
큰 경우 설정할 수 없으며, 과목 순번이 없는 경우에도
설정할 수 없다. 나머지 경우는 적절하게 처리한다.
- GetScore : 과목 순번을 받아 해당_score에서 값을 out 변수에 넣는다.
단, 과목 순번이 적절하지 않으면 정상적인 점수로 간주하지 않는다.

(완성해야 하는 코드)

```
static List<Student> students; //학생 목록 [Student list]
static string path = "c:\\finalA"; //경로 [path]
static string name = "students.dat"; //파일이름 [filename]

참조 0개
static void Main(string[] args)
{
    //성적 평가 기준:중간,기말,출석,과제
    //[Grading criteria: Midterm, Final, Attendance, Homework]
    Student.Subjects = new string[4] { "Midterm", "Final", "Attendance", "Homework" };

    //성적 평가 기준 별 최고 점수
    //[Maximum score by grading criteria]
    Student.Limits = new int[Student.Subjects.Length];
    Student.Limits[0] = 35;
    Student.Limits[1] = 35;
    Student.Limits[2] = 15;
    Student.Limits[3] = 15;

    students = new List<Student>();

    //파일에서 읽어와 학생 정보를 복구한다.
    //[Restore student information from a file.]
    ReadInfo();

    //학생 정보를 입력 받아 처리한다.
    //[Input student information and process it.]
    MainProcess();

    //학생 정보를 파일에 저장한다.
    //[Save student information to a file.]
    WriteInfo();
}
```

이름 : 학번 :

솔루션 : SolTypeA_학번 (SolTypeA_123456)

프로젝트 1 :ProjNo1 (Console App)

프로젝트 2 :ProjNo2 (Console App)

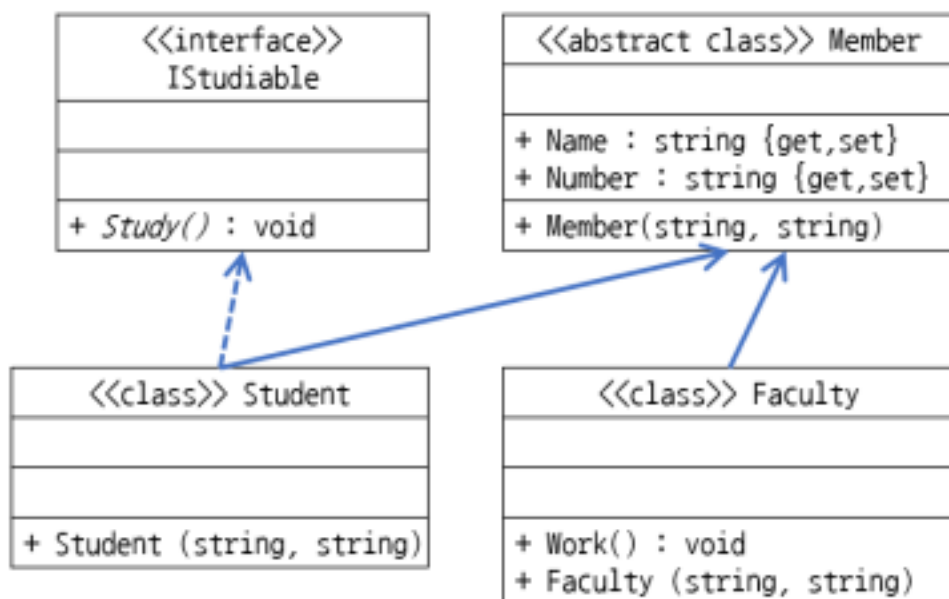
Submission must be saved. After saving, delete the bin and obj folders, compress the solution folder, and then submit it. This exam paper must be submitted.

[UML 표기]

- : private | # : protected | + : public | underbar : static | italic : abstract

[프로젝트 1]

As long as the relationships are correctly established, it's fine. Do not implement the method contents; handle them with empty blocks. If there is a return type, return a well-known default value. Infer the parameters from the given table



[프로젝트 2]

After running the deployed executable program, refer to the UML diagram and the Main() function to write the program.

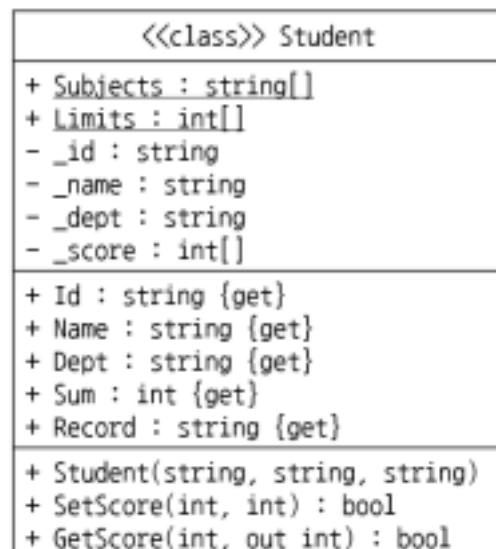
When the program is executed, a file named c:\finalA\students.dat will be created. Upon opening the file, it contains entries in the format StudentID|Name|Department|Grade0,...,GradeN

```
1|John|CS|26,14,12,7
2|Mary|EE|29,6,14,13
3|Tom|ME|5,14,10,10
```

[프로젝트 2]

Student : Class managing student grades

- Subjects: List of subjects for which grades are entered (refer to Main())
- Limits: List of maximum scores for the subjects (refer to Main())
- _id / Id: Student ID
- _name / Name: Name
- _dept / Dept: Department
- _score: List of grades for each subject entered
- Sum: Sum of scores for each subject entered; 0 if no scores are



entered

- Record: String of student information to be saved in the file (refer to the generated file)
- Constructor: Accepts student ID, name, and department to set; creates `_score` based on the list of subjects to be entered.
- SetScore: Sets the score in `_score` for the given subject number and score. However, it cannot be set if the score is less than 0 or greater than the score set in `Limits`, and it cannot be set if the subject number does not exist. Handle other cases appropriately.
- GetScore: Retrieves the value from `_score` for the given subject number and stores it in the out variable. However, if the subject number is not appropriate, it is not considered a valid score.

(완성해야 하는 코드)

```
static List<Student> students; //학생 목록 [Student list]
static string path = "c:\\finalA"; //경로 [path]
static string name = "students.dat"; //파일이름 [filename]

참조 0개
static void Main(string[] args)
{
    //성적 평가 기준:중간,기말,출석,과제
    //[Grading criteria: Midterm, Final, Attendance, Homework]
    Student.Subjects = new string[4] { "Midterm", "Final", "Attendance", "Homework" };

    //성적 평가 기준 별 최고 점수
    //[Maximum score by grading criteria]
    Student.Limits = new int[Student.Subjects.Length];
    Student.Limits[0] = 35;
    Student.Limits[1] = 35;
    Student.Limits[2] = 15;
    Student.Limits[3] = 15;

    students = new List<Student>();

    //파일에서 읽어와 학생 정보를 복구한다.
    //[Restore student information from a file.]
    ReadInfo();

    //학생 정보를 입력 받아 처리한다.
    //[Input student information and process it.]
    MainProcess();

    //학생 정보를 파일에 저장한다.
    //[Save student information to a file.]
    WriteInfo();
}
```